

User: [q_coding_ssm_forward_cpu.ipynb] solve the first coding problem.

I'll help you implement the `unrolled_ssm_forward` function, which performs the forward pass of an SSM using the recursive approach.

Looking at the problem, we need to implement: $h_{t+1} = W h_t + U x_t + b$

where we start with $h_0 = 0$ and compute hidden states for $t = 0, 1, \dots, T-1$.

Here's the implementation:

```
Python
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
         $h_{t+1} = W h_t + U x_t + b$ 
    with initial  $h_0 = 0$ .

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input sequence over T steps
    Returns:
        h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to  $h_{t+1}$  in the usual
    notation).
    """

    #####
    #####
    #                                     TODO: Implement the recurrent pass here
    #

    #####
    #####
    N, T, D = x.shape
    H = W.shape[0]
```

```

# Initialize storage for all hidden states
h_all = torch.zeros(N, T, H)

# Initialize h_0 = 0 for all batch elements
h_t = torch.zeros(N, H)

# Iterate through time steps
for t in range(T):
    # Get input at time t: (N, D)
    x_t = x[:, t, :]

    # Compute h_{t+1} = W @ h_t + U @ x_t + b
    # h_t @ W.T: (N, H) @ (H, H).T = (N, H)
    # x_t @ U.T: (N, D) @ (D, H).T = (N, H)
    h_t = h_t @ W.T + x_t @ U.T + b

    # Store the hidden state
    h_all[:, t, :] = h_t

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
return h_all

```

Explanation:

1. **Extract dimensions:** Get batch size N , time steps T , input dimension D , and hidden dimension H
2. **Initialize storage:** Create `h_all` tensor of shape (N, T, H) to store all hidden states

3. **Initialize hidden state:** Start with $h_0 = 0$ (zero vector of dimension H for each batch element)
4. **Recurrent loop:** For each time step t :
 - Extract input x_t at time t
 - Apply the update rule: $h_{t+1} = W h_t + U x_t + b$
 - Store the computed hidden state in $h_{all}[:, t, :]$
5. **Matrix multiplication:** Use `@` operator with transposed weight matrices to handle batched operations efficiently

The key insight is that we're computing the state sequentially, where each new state depends on the previous one, which is the defining characteristic of the recurrent approach.

Basically identical to the staff solution, save for the usage of the `@` symbol for matmul. Included some nice explanation of each component.

User: now the next part

I'll help you implement the convolution-based approach for the SSM forward pass. This involves implementing both `make_conv_kernel` and `conv_ssm_forward`.

Python

```
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use
    when implementing
    the ssm forward pass using conv1d.

    Args:
        W: (H, H) weight matrix
        T: scalar

    Returns:
        kernel_for_conv: (H, H, T) tensor
    """
```



```
#####
#####
    return kernel_for_conv
```

This is mostly the same, but lacks the `torch.flip` in the staff solution that reverses the kernel into the proper causal format.

```
Python
def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b

    s = s.permute(0, 2, 1)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

    #####
    #####
```

```

#                                     TODO: Implement the convolution
here                                #

#####

# Reshape s for convolution: (N, H, T)
# Reshape kernel for F.conv1d: need (H, H, T)

# We need to apply convolution for each batch element
# For each output channel h_i, we convolve input channels
with kernel[h_i, :, :]

# Use conv1d with groups or loop over batch
h_conv = torch.zeros(N, H, T)

for n in range(N):
    # For each batch element, apply convolution
    # s[n] has shape (H, T)
    # We need to convolve s[n] with kernel to get h[n]

    for h_out in range(H):
        # For each output hidden dimension
        # Convolve all input dimensions with corresponding
kernel weights
        for h_in in range(H):
            # kernel[h_out, h_in, :] has shape (T,)
            # s[n, h_in, :] has shape (T,)
            # We need causal convolution
            for t in range(T):
                # At time t, we sum over all previous times
(0 to t)

                for tau in range(t + 1):
                    h_conv[n, h_out, t] += kernel[h_out,
h_in, t - tau] * s[n, h_in, tau]

```

```

# Permute back to (N, T, H)
h_all = h_conv.permute(0, 2, 1)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
    return h_all

```

This is functionally the same as the staff solution.

Explanation:

`make_conv_kernel` function:

- Creates a kernel of shape (H, H, T) where `kernel[:, :, t]` represents W^t (W to the power of t)
- Starts with $W^0 = I$ (identity matrix)
- Iteratively computes $W^1, W^2, \dots, W^{T-1}$ by matrix multiplication
- This kernel will be used in the convolution to capture the recurrent dependencies

`conv_ssm_forward` function:

- First projects the input: $s = x @ U.T + b$ to get shape (N, T, H)
- Permutes to (N, H, T) for convolution operations
- Builds the convolution kernel
- Applies causal convolution: at each time t , the output depends only on inputs from times 0 to t
- The convolution implements: $h_t = \sum_{\tau=0}^t W^{t-\tau} s_{\tau}$
- Permutes back to (N, T, H) format

This convolution-based approach computes all time steps in parallel (conceptually), unlike the sequential recurrent approach, which is the key insight for potential speedup.

User: ``

Unrolled h(t):

```
tensor([[[[-1.2720e-02, 1.7478e-01, 1.1702e-01, -1.8475e-01],
          [ 4.3287e-02, 1.7190e-01, 7.0203e-02, 2.0290e-01],
          [ 5.2788e-03, 1.1071e-01, 4.4593e-02, 2.2496e-01],
          [-3.0554e-01, 3.6337e-01, -1.6552e-01, -5.3192e-01],
          [ 1.0813e-01, 5.4074e-01, 6.1866e-02, -1.0820e-01],
          [-2.6562e-02, 3.5316e-01, 2.7978e-02, 4.6655e-02],
          [-4.7121e-02, -9.5886e-02, 3.1475e-02, 2.7231e-01],
          [-1.4480e-01, 2.0062e-01, -1.8574e-01, 1.4919e-01]],

        [[ 2.0503e-01, 1.1027e-01, 2.8894e-01, 1.3814e-01],
          [ 4.6919e-04, 3.3735e-01, 7.8319e-02, 3.0716e-02],
          [-2.1996e-02, 7.8201e-02, 8.5306e-02, 6.4196e-02],
          [ 9.9243e-03, 2.1734e-01, 4.4236e-03, 1.9512e-01],
          [-1.0414e-01, 2.8412e-01, -8.0947e-02, 1.0402e-02],
          [-1.3561e-01, 4.3784e-01, -1.1956e-01, -2.1006e-01],
          [ 2.1366e-01, 6.7647e-02, 2.8377e-01, 2.1382e-01],
          [ 1.3880e-02, 1.7214e-01, 1.7150e-01, -3.1018e-02]]]])
```

Conv-based h(t):

```
tensor([[[[-1.2720e-02, 1.7478e-01, 1.1702e-01, -1.8475e-01],
          [ 4.3287e-02, 1.7190e-01, 7.0203e-02, 2.0290e-01],
          [ 5.2788e-03, 1.1071e-01, 4.4593e-02, 2.2496e-01],
          [-3.0554e-01, 3.6337e-01, -1.6552e-01, -5.3192e-01],
          [ 1.0813e-01, 5.4074e-01, 6.1866e-02, -1.0820e-01],
          [-2.6562e-02, 3.5316e-01, 2.7978e-02, 4.6655e-02],
          [-4.7121e-02, -9.5886e-02, 3.1475e-02, 2.7231e-01],
          [-1.4480e-01, 2.0062e-01, -1.8574e-01, 1.4919e-01]],

        [[ 2.0503e-01, 1.1027e-01, 2.8894e-01, 1.3814e-01],
          [ 4.6919e-04, 3.3735e-01, 7.8319e-02, 3.0716e-02],
          [-2.1996e-02, 7.8201e-02, 8.5306e-02, 6.4196e-02],
          [ 9.9243e-03, 2.1734e-01, 4.4236e-03, 1.9512e-01],
          [-1.0414e-01, 2.8412e-01, -8.0947e-02, 1.0402e-02],
          [-1.3561e-01, 4.3784e-01, -1.1956e-01, -2.1006e-01],
          [ 2.1366e-01, 6.7647e-02, 2.8377e-01, 2.1382e-01],
          [ 1.3880e-02, 1.7214e-01, 1.7150e-01, -3.1018e-02]]]])
```

Max absolute difference: 5.960464477539063e-08

...

now answer the questions.

Great! The implementations are working correctly with a very small numerical difference ($\approx 6e-8$), which is due to floating-point precision. Now let me answer the questions:

Question 1: Maximum Absolute Difference

Answer: The maximum absolute difference observed between the two implementations is approximately 5.96×10^{-8} (or 0.0000000596). This tiny difference is due to floating-point arithmetic precision and demonstrates that both implementations are numerically equivalent.

Correct.

Question 2: Number of Operations in Recurrence-Based Implementation

Answer: The recurrence-based implementation performs operations at each time step sequentially:

For each time step t (total of T steps):

- Matrix multiplication $h_t @ W.T$: $O(N \times H \times H)$ operations
- Matrix multiplication $x_t @ U.T$: $O(N \times H \times D)$ operations
- Addition of bias: $O(N \times H)$ operations

Total complexity: $O(T \times N \times H \times (H + D))$

Or more specifically: $O(T \times N \times H^2) + O(T \times N \times H \times D)$

The key characteristic is that operations scale **linearly with T** because we must process each time step sequentially.

Claude includes an extra factor of N to account for the batch size, but this is otherwise correct.

Question 3: Number of Operations in Convolution-Based Implementation

Answer: The convolution-based implementation has two main phases:

1. Kernel Construction (one-time cost):

- Computing W^t for $t = 0$ to $T-1$: $O(T \times H^3)$ operations
 - Each matrix multiplication $W^t = W^{(t-1)} \times W$ costs $O(H^3)$

2. Convolution Operation:

- The causal convolution for each batch element and output dimension requires summing over all input dimensions and previous time steps
- For each of N batch elements, H output dimensions, and T time steps:
 - At time t , sum over H input dimensions and $(t+1)$ previous times
 - Total: $O(N \times H^2 \times T^2)$ operations

Total complexity: $O(T \times H^3) + O(N \times H^2 \times T^2)$

For large T , this is dominated by $O(N \times H^2 \times T^2)$, which scales **quadratically with T** .

Again, Claude includes an extra factor of N to account for the batch size, but this is otherwise correct.

Question 4: Trade-off Between the Two Implementations

Answer:

Recurrence-based approach:

- **Pros:**
 - Linear scaling with T : $O(T \times N \times H^2)$
 - Lower memory footprint (only stores current hidden state)
 - Better for very long sequences
- **Cons:**
 - Inherently sequential - cannot be parallelized across time
 - Slower for short/medium sequences on parallel hardware

Convolution-based approach:

- **Pros:**
 - Highly parallelizable across time steps
 - Can leverage optimized convolution operations (especially on GPUs)
 - Faster for short to medium sequences when hardware parallelism is available
- **Cons:**
 - Quadratic scaling with T : $O(T^2)$
 - Higher memory requirements (stores kernel of size $H \times H \times T$)
 - Becomes slower than recurrence for long sequences

Key Trade-off:

- For **short sequences** (small T) with **parallel hardware**: convolution is faster despite higher complexity

- For **long sequences** (large T): recurrence is better due to linear vs quadratic scaling
 - The crossover point depends on hardware capabilities (CPU vs GPU) and the specific values of H, D, and N
-

Looking at the CPU runtime plot for H=40 with diagonal matrices, this reveals some very interesting and somewhat unexpected behavior:

Analysis of CPU Diagonal Matrix Runtime Plot

Key Observations:

1. Unrolled (Recurrence) behavior:

- Shows expected linear growth with T
- Starts at ~0.001 sec for T=8
- Grows to ~0.05 sec for T=512
- Clean, predictable $O(T)$ scaling

2. Diagonal-Conv behavior (surprising!):

- Shows a **peak at T=256** (~0.03 sec)
- **Faster at T=8 and T=512** (~0.001 sec) than at T=256
- Creates an inverted-U shape rather than monotonic growth
- This is counterintuitive given $O(T^2)$ or $O(T)$ expected complexity

Why does Diagonal-Conv show this strange pattern?

This unusual behavior is likely due to several CPU-specific factors:

1. Cache effects:

- At T=256, the kernel size and intermediate data may exceed L2/L3 cache
- Small T (8, 32): Everything fits in cache → very fast
- Medium T (256): Spills out of cache → cache misses → slower
- Large T (512): Different memory access patterns or compiler optimizations kick in

2. Vectorization efficiency:

- CPU SIMD instructions work best with certain data sizes
- T=256 might hit an "awkward" size for vectorization
- Powers of 2 like 512 might align better with vector register sizes (256-bit AVX2, 512-bit AVX-512)

3. Loop unrolling and compiler optimizations:

- Compilers optimize differently for different loop sizes

- $T=512$ might trigger different optimization strategies
- The nested loop structure in our implementation may benefit from specific T values

4. Memory layout and striding:

- At $T=256$, memory access patterns might be sub-optimal
- Larger T might benefit from better memory prefetching

Comparison with unstructured case expectations:

In the unstructured matrix case on CPU, we'd expect:

- Recurrence: Linear $O(T)$ growth
- Convolution: Quadratic $O(T^2)$ growth, becoming much slower

With diagonal matrices:

- Recurrence: Still linear, but $\sim 10\times$ faster due to element-wise ops
- Diagonal-Conv: Should theoretically be $O(T)$ with optimized depthwise, but shows complex performance characteristics on CPU

Key insight for Question 7:

The diagonal structure does make both methods faster, but on CPU, the convolution approach shows **unpredictable performance** due to low-level hardware interactions. The recurrence method, while potentially slower in absolute terms, shows **consistent, predictable scaling**.

This suggests that:

- **On CPU with diagonal matrices:** Recurrence might be preferable for its predictability
- **On GPU with diagonal matrices:** Convolution should dominate due to better parallel hardware support
- The performance story is highly hardware-dependent

This plot illustrates why modern SSM implementations focus on GPU deployment—the parallel hardware provides more predictable benefits from structural constraints like diagonal matrices.

This answer is very detailed, and gets into the nitty-gritty of how CPU vs GPU would work. It even gets down to the CPU SIMD instructions. I really like this answer!

start to solve this notebook, starting from Introducing structure: Diagonal Weight Matrices

I'll help you solve the GPU notebook starting from the diagonal weight matrices section. Let me implement the required functions.

1. make_diag_depthwise_kernel function

Python

```
def make_diag_depthwise_kernel(W, T):  
    """  
    Construct a depthwise 1D conv kernel for W with shape (H,H).  
    W is a diagonal matrix.  
  
    Args:  
        W: (H, H) diagonal matrix  
        T: (int) number of time steps  
  
    Return:  
        kernel of shape (H, 1, T), which can be used in depthwise  
        convolution  
    """  
  
    #####  
    #####  
  
    #                                TODO: Implement the kernel here  
    #  
  
    #####  
    #####  
  
    H = W.shape[0]
```

```
# Extract diagonal elements

diag_vals = torch.diag(W) # Shape: (H,)

# Initialize kernel

kernel = torch.zeros(H, 1, T)

# For depthwise convolution, each channel is independent

# kernel[i, 0, t] = diag_vals[i]^t

for i in range(H):

    w_i = diag_vals[i]

    w_power = 1.0 # w_i^0

    for t in range(T):

        kernel[i, 0, t] = w_power

        if t < T - 1:

            w_power = w_power * w_i

#####
#####

# END OF YOUR CODE

#
```

```
#####
#####

return kernel
```

2. **diag_conv_ssm_forward** function

Python

```
def diag_conv_ssm_forward(W, U, b, x):

    """

    Convolution-based forward pass for an RNN with W

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ .
```

Args:

W: (H, H) [diagonal in practice]

U: (H, D)

b: (H,)

x: (N, T, D) => N = batch, T = time steps, D = input dim

Returns:

```

        h_all: (N, T, H)

    """

    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b

    s = s.permute(0, 2, 1)  # (N,H,T)

    # Build kernel (H,1,T) for depthwise conv
    kernel = make_diag_depthwise_kernel(W, T)

#####
#####

    #                                TODO: Implement the convolution
    here                                #

    #                                Hint: Use `groups` argument
    #

#####
#####

    # Depthwise convolution: each input channel is convolved
    independently

```


[illegible]

```
return h_all
```

3. **diag_unrolled_ssm_forward** function

Python

```
def diag_unrolled_ssm_forward(W, U, b, x):  
    """  
  
    Forward pass for:  
  
         $h_{t+1} = W h_t + U x_t + b$   
  
    but W is diagonal, i.e.  $W = \text{diag}(\text{diag}W)$ .  
  
    Args:  
  
        W: (H, H) diagonal matrix  
  
        U: (H, D)  
  
        b: (H,)  
  
        x: (N, T, D) => batch=N, time steps=T, input dim=D  
  
    Returns:  
  
        h_all: (N, T, H)  
  
    """  
  
    #####  
    #####
```

```

#          TODO: Implement the optimized recurrent pass
here      #

#####
#####

N, T, D = x.shape

H = W.shape[0]

# Extract diagonal values for efficient computation
diag_vals = torch.diag(W) # Shape: (H,)

# Initialize storage for all hidden states
h_all = torch.zeros(N, T, H)

# Initialize h_0 = 0 for all batch elements
h_t = torch.zeros(N, H)

# Iterate through time steps
for t in range(T):

    # Get input at time t: (N, D)

    x_t = x[:, t, :]

```

```

        # For diagonal W,  $h_{t+1} = \text{diag\_vals} * h_t + U @ x_t + b$ 

        # Element-wise multiplication instead of matrix
multiplication

        h_t = diag_vals * h_t + x_t @ U.T + b

    # Store the hidden state

    h_all[:, t, :] = h_t

#####
#####

#                                     END OF YOUR CODE
#

#####
#####

    return h_all

```

Explanation

Key optimizations for diagonal matrices:

1. **make_diag_depthwise_kernel:**

- For a diagonal W , each hidden dimension evolves independently
- We only need to store powers of each diagonal element: $[1, w_i, w_i^2, \dots, w_i^{(T-1)}]$
- Output shape is $(H, 1, T)$ for depthwise convolution

2. `diag_conv_ssm_forward`:

- Uses `F.conv1d` with `groups=H` for depthwise convolution
- Each of the H channels is processed independently with its own 1D filter
- Pads the input on the left with $T-1$ zeros to implement causal convolution
- This is much more efficient than the full convolution since we avoid $H \times H$ matrix operations

3. `diag_unrolled_ssm_forward`:

- Extracts diagonal values once: `diag_vals = torch.diag(W)`
- Replaces matrix multiplication `h_t @ W.T` with element-wise multiplication `diag_vals * h_t`
- This reduces complexity from $O(H^2)$ to $O(H)$ per time step

Complexity comparison:

- **Full matrix:** $O(T \times N \times H^2)$ for recurrence, $O(T^2 \times N \times H^2)$ for convolution
- **Diagonal matrix:** $O(T \times N \times H)$ for recurrence, $O(T \times N \times H)$ for depthwise convolution

The diagonal constraint provides significant speedups for both methods!

This is mostly correct, but Claude's implementation of `diag_conv_ssm_forward` led to a slight difference, due to a slight issue where the code attempts to implement padding *twice*, which leads to the large difference that we observe.

...

Unrolled $h(t)$:

```
tensor([[[[-0.0563, 0.0190, 0.0105, -0.1580],
          [ 0.1031, -0.0293, 0.1256, -0.2023],
          [ 0.0925, -0.0149, 0.0961, -0.4178],
          [-0.0540, -0.0726, 0.0494, -0.2727],
          [-0.4944, -0.1865, -0.0981, 0.3956],
          [ 0.0738, -0.1133, 0.1744, 0.1411],
          [-0.1444, -0.1421, 0.0544, 0.0483],
          [-0.2369, -0.1165, 0.0221, 0.4350]],

        [[-0.3021, -0.0758, -0.0509, 0.3044],
          [ 0.1761, -0.0469, 0.1658, -0.3138],
          [-0.3661, -0.1767, -0.0704, -0.1081],
          [-0.5248, -0.1941, -0.1096, 0.4271],
          [ 0.1389, -0.0366, 0.1629, -0.0441],
          [-0.2679, -0.1284, -0.0054, 0.2542],
          [-0.5545, -0.1520, -0.1616, 0.2249],
          [-0.0282, -0.0786, 0.0734, -0.1926]]], device='cuda:0')
```

Conv-based h(t):

```
tensor([[[[-5.6252e-09, 1.8970e-09, 1.0461e-09, -1.5795e-08],
          [-4.5378e-08, 1.5848e-08, 2.2913e-08, -1.7660e-07],
          [-4.4556e-07, 1.5728e-07, 2.3748e-07, -1.8057e-06],
          [-4.4619e-06, 1.5657e-06, 2.3788e-06, -1.8080e-05],
          [-4.4668e-05, 1.5639e-05, 2.3778e-05, -1.8076e-04],
          [-4.4666e-04, 1.5638e-04, 2.3780e-04, -1.8076e-03],
          [-4.4667e-03, 1.5638e-03, 2.3780e-03, -1.8076e-02],
          [-4.4667e-02, 1.5638e-02, 2.3780e-02, -1.8076e-01]],

        [[-3.0213e-08, -7.5794e-09, -5.0924e-09, 3.0438e-08],
          [-2.8150e-07, -7.9722e-08, -3.3830e-08, 2.6996e-07],
          [-2.8534e-06, -8.1442e-07, -3.4700e-07, 2.6919e-06],
          [-2.8583e-05, -8.1618e-06, -3.4802e-06, 2.6963e-05],
          [-2.8581e-04, -8.1620e-05, -3.4785e-05, 2.6962e-04],
          [-2.8581e-03, -8.1621e-04, -3.4785e-04, 2.6962e-03],
          [-2.8581e-02, -8.1621e-03, -3.4785e-03, 2.6962e-02],
          [-2.8581e-01, -8.1621e-02, -3.4785e-02, 2.6962e-01]]]],
       device='cuda:0')
```

Max absolute difference: 0.6157543063163757

...

The outputs show a significant difference! The convolution-based implementation is producing values that are orders of magnitude smaller than expected. This suggests an issue with how we're applying the convolution. Let me fix the implementations:

Fixed `diag_conv_ssm_forward` function

Python

```
def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with W

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
    but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ .

    Args:
        W: (H, H) [diagonal in practice]
```

```

    U: (H, D)
    b: (H,)
    x: (N, T, D) => N = batch, T = time steps, D = input dim

Returns:
    h_all: (N, T, H)
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b

    s = s.permute(0, 2, 1)  # (N,H,T)

    # Build kernel (H,1,T) for depthwise conv
    kernel = make_diag_depthwise_kernel(W, T)

#####
#####
    #                                TODO: Implement the convolution
here                                #
    #                                Hint: Use `groups` argument
#

#####
#####

    # We need to implement causal convolution
    # At time t, output depends on inputs from time 0 to t

    # Method: manually compute the causal convolution
    h_all_list = []

    for t in range(T):
        # For each output time t, we need to sum contributions
        from times 0 to t

```

```

        #  $h[t] = \sum_{\tau=0}^t \text{kernel}[:, 0, t-\tau] * s[:, :, \tau]$ 
        tau]

        h_t = torch.zeros(N, H, device=x.device)

        for tau in range(t + 1):
            # kernel[:, 0, t-tau] has shape (H,)
            # s[:, :, tau] has shape (N, H)
            h_t += kernel[:, 0, t - tau].unsqueeze(0) * s[:, :, tau]

        h_all_list.append(h_t)

    # Stack all time steps: (T, N, H) -> (N, T, H)
    h_all = torch.stack(h_all_list, dim=1)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
    return h_all

```

Actually, let me provide a more efficient implementation using proper convolution padding:

Better `diag_conv_ssm_forward` function

```

Python
def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with W

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

```


but W is diagonal, so $h_{t+1}(i) = \text{diag}W[i] \cdot h_t(i) + [U x_t + b](i)$.

Args:

W : (H, H) [diagonal in practice]

U : (H, D)

b : (H,)

x : (N, T, D) => N = batch, T = time steps, D = input dim

Returns:

h_{all} : (N, T, H)

"""

N, T, D = x.shape

H = W.shape[0]

s = x @ U.T + b

s = s.permute(0, 2, 1) # (N,H,T)

Build kernel (H,1,T) for depthwise conv

kernel = make_diag_depthwise_kernel(W, T)

#####

#####

#

TODO: Implement the convolution

here

#

#

Hint: Use `groups` argument

#

#####

#####

Flip the kernel for proper convolution (conv1d does cross-correlation)

kernel_flipped = torch.flip(kernel, dims=[2])

```

# Pad on the left for causal convolution
s_padded = F.pad(s, (T - 1, 0))

# Apply depthwise convolution with groups=H
h_conv = F.conv1d(s_padded, kernel_flipped, groups=H,
padding=0)

# Permute back to (N, T, H)
h_all = h_conv.permute(0, 2, 1)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
return h_all

```

The key issue was that `F.conv1d` computes cross-correlation (not true convolution), so we need to flip the kernel. Also, the padding and indexing need to be correct for causal convolution.

Let me provide the most straightforward implementation that matches the recurrent formula exactly:

Simplest correct implementation

```

Python
def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with W

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
    but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ .

```

Args:

W: (H, H) [diagonal in practice]

U: (H, D)

b: (H,)

x: (N, T, D) => N = batch, T = time steps, D = input dim

Returns:

h_all: (N, T, H)

"""

N, T, D = x.shape

H = W.shape[0]

s = x @ U.T + b # (N, T, H)

s = s.permute(0, 2, 1) # (N, H, T)

Build kernel (H,1,T) for depthwise conv

kernel = make_diag_depthwise_kernel(W, T)

#####

here #

Hint: Use `groups` argument
#

#####

Initialize output

h_conv = torch.zeros(N, H, T, device=x.device)

For each time step t, compute h[t] as sum of kernel * s up
to time t

for t in range(T):

```

        for tau in range(t + 1):
            # kernel[:, 0, t-tau] * s[:, :, tau]
            h_conv[:, :, t] += kernel[:, 0, t - tau] * s[:, :,
tau]

        # Permute back to (N, T, H)
        h_all = h_conv.permute(0, 2, 1)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
        return h_all

```

This implementation explicitly computes the causal convolution and should match the recurrent version exactly. While it's not as efficient as using `F.conv1d`, it's clearer and more reliable for understanding what's happening.

Claude iterated through a couple different iterations of the code before settling on a correct answer. It was interesting to see this sort of chain of thought within the same message, unprompted. It was able to completely fix the issue without much help. It also seems to treat clarity as a higher priority, after having made one mistake.

Perfect! Now the implementations match with a negligible difference ($\approx 3e-8$). Now let me answer the questions.

Ah! This is the **GPU diagonal matrix runtime plot** (H=40), and it tells a completely different story from the CPU plot!

Analysis of GPU Diagonal Matrix Runtime Plot

Key Observations:

1. **Diagonal-Conv (orange line):**

- **Essentially flat** across all T values (~0.001 sec)
 - No visible increase from T=8 to T=512
 - Maintains consistent, extremely fast performance
 - **This is the ideal behavior we hoped for!**
2. **Unrolled/Recurrence (blue line):**
- Shows clear **linear growth** with T
 - T=8: ~0.013 sec
 - T=512: ~0.057 sec
 - Predictable $O(T)$ scaling
 - About **4-50× slower** than diagonal-conv depending on T

Why is GPU diagonal-conv so efficient?

1. **Highly optimized depthwise convolution kernels:**
 - GPU libraries (cuDNN, PyTorch) have specialized kernels for depthwise conv
 - With **groups=H**, each channel processes completely independently
 - Perfect parallelization with no inter-channel dependencies
2. **Constant work per output element:**
 - Each output at time t only depends on inputs up to time t
 - GPU can compute all T time steps in parallel across different threads
 - The apparent $O(T^2)$ complexity is hidden by massive parallelism
3. **Memory bandwidth optimization:**
 - Depthwise operations have very predictable memory access patterns
 - GPUs can effectively pipeline and prefetch data
 - Coalesced memory accesses maximize bandwidth utilization
4. **No cache issues like CPU:**
 - GPU has different memory hierarchy optimized for this workload
 - Thousands of threads hide memory latency through occupancy
 - No "cache thrashing" like we saw on CPU at T=256

Comparison: CPU vs GPU Diagonal Matrix Performance

Metric	CPU (from previous plot)	GPU (current plot)
Conv scaling with T	Erratic (peak at T=256)	Flat (no growth)

Conv predictability	Poor	Excellent
Conv vs Recurrence	Competitive/variable	Conv dominates
Recurrence scaling	Linear (predictable)	Linear (predictable)
Best method	Depends on T	Always Conv

Updated Answer to Question 7

With the GPU plot in hand:

Observations with diagonal weight matrices on GPU:

- Diagonal convolution achieves near-constant time:**
 - Runtime stays at ~0.001 sec regardless of T
 - This suggests the GPU implementation effectively achieves $O(1)$ time with respect to T (up to memory limits)
 - The theoretical $O(T)$ or $O(T^2)$ complexity is completely hidden by parallelism
- Diagonal convolution dramatically outperforms recurrence:**
 - At T=512, convolution is ~57× faster
 - The gap widens as T increases
 - Recurrence cannot parallelize across time, so it scales linearly
- This explains why modern SSMs use diagonal/structured matrices:**
 - Models like S4, Mamba, and others impose diagonal or low-rank structure
 - On GPU, this enables **parallelizable convolution** that's vastly faster than recurrence
 - During training (where you process entire sequences), convolution mode is used
 - During inference (autoregressive generation), efficient recurrence formulations can be used

Differences from unstructured case:

Aspect	Unstructured (GPU)	Diagonal (GPU)
Conv performance	Good but grows with T	Essentially constant
Conv vs Recurrence	Competitive	Conv wins decisively
Practical use	Limited by T^2 scaling	Scales to very long sequences
Real-world applicability	Not practical for $T > 1000$	Can handle $T > 10,000+$

Conclusion:

The diagonal structure is the key innovation that makes modern SSMs practical. On GPU:

- It reduces the convolution to depthwise operations that are massively parallel
- Runtime becomes nearly independent of sequence length (up to memory limits)
- This enables training on sequences that would be impossible with full matrix SSMs

This is why papers like S4 and Mamba emphasize structured matrices—it's not just about reducing parameters, it's about unlocking hardware-efficient parallel computation that makes these models competitive with Transformers for long sequences.

This answer is correct, and is very, very detailed as well. I really like it because it gives a really deep insight into caching on CPU vs GPU.